

한입지도 — 서비스 마스터 가이드

예산 안에서 오늘 뭘 먹을지, 검증된 데이터로 정해주는 서비스. 이 문서 하나로 서비스 전부와 적대적 심사 질문을 방어한다.

LIVE · sixsense.askewly.comCODE · github.com/lumatic2/upstage-sixsense

기준 · 2026-07-24 실측

00 30초 안에 이 서비스를 설명한다면

성균관대 명륜캠퍼스 주변에서, 오늘 가진 예산으로 먹을 수 있는 곳을 알려주는 서비스다. 동네 식당의 가격은 웹에서 믿을 수가 없다 — 소스마다 값이 다르고 언제 값인지도 없다. 그래서 우리는 **메뉴판 사진을 직접 찍어 Upstage로 읽고, 사람이 검수한 가격만** 데이터로 만들었다. 그 위에서 예산·도보 시간으로 검색하고, 모델이 쓴 추천 이유는 **다른 모델이 데이터와 대조해 검증한다**. 범용 챗봇은 답을 **지어내고**, 한입지도는 검수된 데이터에서 답을 **찾는다**.

이 한 단락이 발표의 전부다. 아래는 이걸 심사 질문 앞에서 지켜내기 위한 살들이다.

01 먼저 외울 숫자 (입에서 같은 값이 나와야 한다)

50곳

검수 통과 식당 (수집 총 53)

719개

파싱이 만든 메뉴 후보

580개

검수 통과·화면에 나가는 메뉴

0건

검수 대기 (밀린 것 없음)

76.4%

파싱 정확도 (사진5·메뉴72 채점)

4곳

매일 크롤링하는 학식당

\$0.08

정확도 측정 전체 비용 (8회 호출)

4자리

Upstage 제품이 일하는 위치

더 물어보면 나오는 값

답

검수 3상태

확인 **580** · 제외 **139** · 대기 **0** (합 719)

화면에 나가는 메뉴 분해

본메뉴 **428** · 사이드 **152**

본메뉴 가격

중앙값 **12,000원** (2,500 ~ 75,000원)

정확도 향상 궤적

0% → 62.5% → 76.4% (**API 재호출 없이** 후처리 규칙만 2번 수정)

사진 상태별 정확도

인쇄체·고해상 **100%** / 손글씨 64% / 저해상 20%

Solar 응답 속도

실측 **0.7~1.3초**, 타임아웃 시 정규식 폴백

숫자 규칙 — 절대 어기지 말 것

슬라이드·화면(/about.html 자동 표시)·내 입에서 나오는 숫자가 **전부 같아야 한다**. 헛갈리면 50 · 719 · 580 · 76.4% 이 네 개만 확실히 붙잡는다.

⚠ **후보 719는 라이브에서 늘 수 있다** — 제보가 계속 들어와 어젯밤 기준 시트는 ~734. 슬라이드·입은 **719로 통일**하되, 심사위원이 /about.html 에서 더 큰 수를 봐도 당황 말 것("후보는 제보로 늘고, 검수 통과 580·대기 0이 고정 앵커"). 리허설에서 라이브 값 1회 확인.

02 문제 정의 — 여기서 프레임을 틀리면 다 무너진다

타깃: 성균관대 명륜캠퍼스 학생. 점심·저녁마다 "얼마 있는데 뭐 먹지"를 하루 두 번 반복하는 사람.

문제: 동네 식당의 메뉴·가격은 **웹에서 믿을 수가 없다**. 블로그·리뷰에 가격이 있긴 한데 소스마다 값이 다르고 **언제 값인지가 없다**. 실측 예: 세아 짜장면이 다이닝코드엔 6,000원, 지난달 방문 블로그엔 7,000원. 지도 앱엔 상호·위치만 있다. 지금 가격의 유일한 근거는 **가게 벽에 붙은 메뉴판 사진**이다.

그래서 ChatGPT에 "명륜동에서 8천원으로 뭐 먹지"라고 물으면 이렇게 끝난다 — **"정확한 가격은 매장에 문의하세요."** 이걸 모델이 부족해서가 아니라 **믿고 읽을 데이터가 없어서**다. 문제는 모델이 아니라 데이터 쪽에 있다.

★ 정확한 프레임 — 이 표현만 쓴다

"웹에 가격이 **없다**"고 말하면 심사위원이 바로 반박한다("검색하면 나오는데요?"). 실제로 검색하면 나온다.

✓ 올바른 주장 두 개: ① 가격이 없는 게 아니라 **어느 게 지금 값인지 판별이 안 된다**(6,000 vs 7,000 공존). ② **예산으로 50곳을 필터·비교하는 질의는 실시간 검색으로 안 된다** — 구조화된 데이터가 있어야 한다. 그게 우리가 만든 것.

03 해법 — 데이터를 직접 만든다 (vs 범용 LLM)

메뉴판 사진을 모아 Upstage Document Parse로 읽고, **사람이 검수한 것만** 서비스에 올린다. 그 위에서 예산·도보 시간으로 검색한다. 이게 범용 챗봇이 구조적으로 못 하는 일을 하는 방식이다.

항목	범용 LLM	한입지도
가격 출처	학습 데이터 (시점 불명)	메뉴판 사진 + 촬영일 + 검수자
없는 정보	그렇듯하게 지어냄	빈칸으로 둠
검증	없음	다른 모델이 근거 판정
예산 검색	문장으로만	실제 가격 데이터로 필터

한 문장 차별성: 범용 LLM은 답을 **지어내고**, 한입지도는 검수된 데이터에서 답을 **찾는다**.

04 데이터 파이프라인 배점 20점 · 최대 무게

핵심 주장: 입력은 세 갈래인데, 나가는 문은 하나다. 어느 경로로 들어왔든 **사람이 확인한 것만** 화면에 나간다.

메뉴판 사진 → Upstage Document Parse
학식 웹페이지 → 매일 06:10 자동 크롤링
가게 주소 → 주소 → 카카오 좌표 변환

└─▶ 사람 검수 ─▶ 한입지도

단계별 (5단계)

단계	하는 일
① 수집	명륜동·혜화 식당을 직접 돌며 메뉴판 촬영. 출처와 촬영일을 함께 저장.
② 파싱	Document Parse에 <code>ocr=force</code> , <code>coordinates=true</code> 로 요청 → 메뉴명·가격 추출.
③ 검수	사람이 원본 사진과 한 줄씩 대조. 검수=확인 인 행만 노출 . 이 게이트는 API 레벨이라 우회 경로가 없다.
④ 보강	주소로 좌표를 채우고(도보 계산용), 학식은 매일 06:10 크롤링(vercel cron).
⑤ 서비스	예산·도보·태그로 필터 → Solar가 추천 이유 생성 → 다른 모델이 근거 판정.

왜 학식은 파싱하지 않나 (반드시 알아둘 것)

학교 식단은 이미 표로 정리된 HTML이다. 여기 사진 파싱을 없으면 비용은 늘고 정확도는 떨어진다.
비정형=Parse, 정형=크롤러 — 이 구분 자체가 우리의 Upstage 활용 설계다. "쓸 수 있다고 다 쓰지 않았다"가 오히려 점수다.

05 Upstage 적용

배점 20점

네 자리에서 쓴다. 데이터가 들어올 때 둘, 사용자에게 나갈 때 둘.

제품 · 위치	무슨 일	왜 이 모델
Document Parse 들어올 때	메뉴판 사진에서 메뉴명·가격을 읽음 (ocr=force , coordinates=true)	지금 가격의 근거는 메뉴판 사진뿐 — 웹 가격은 소스마다 어긋남
Solar solar-pro2 들어올 때	제보 사진에서 가게 이름을 찾음	사람이 타이핑할 칸을 줄임. 이름 없으면 빈칸을 돌려주게 못 박음
Solar solar-pro2 나갈 때	사용자 문장에서 예산·도보·태그 추출 + 추천 이유 작성 (한 번의 왕복, 구조화 출력)	"둘이서 2만원"→1인 1만원 나눠셈을 solar-mini는 틀림
Solar solar-mini 나갈 때	pro2가 쓴 추천 이유를 데이터와 대조해 채점	쓴 모델이 스스로 채점하면 검증이 아니라서 다른 모델에 맡김

구조화 출력 예시 (외워두면 시연·질문에 강함)

입력: "8천 원 이하 10분 거리 이내 혼밥"
출력: { "budget": 8000, "walkMax": 10, "tags": ["혼밥"],
"reason": "8천원 이하 예산, 10분 이내 거리, 혼밥 태그 조건 명시됨", "source": "solar" }

★ 정직성 각주 — 이걸 모르면 함정 질문에 무너진다
"전용 Groundedness Check 모델을 쓴 거냐?" → 아니다. 전용 모델은 2026-07 현재 계정 모델 목록에서 제공 종료됐다(/v1/models 실측). 같은 계약(context-answer 판정)을 생성과 분리된 solar-mini 판정자로 구현했다. 정확한 서사는 "제품 3종"이 아니라 "Solar를 생성·검증 2역할로 분리 활용"이다.

Upstage 공식 서사 ① = "모델이 아니라 문서 구조화가 병목". 풀어 쓰면 — 기업이 AI를 문서에 쓸 때 진짜 어려운 건 똑똑한 모델이 아니라, 어지러운 문서(사진·스캔·PDF)를 AI가 쓸 수 있는 깨끗한 데이터로 바꾸는 일이라는 뜻이다. 우리는 그 어려운 지점(메뉴판 사진→가격 데이터)을 실물로 해냈다.
Upstage 공식 서사 ② = "할루시네이션 없는 정확성(Precision)"(킵오프 p17). 우리 환각 방지 4겹(사람 검수·근거 판정 포함)이 이 가치와 그대로 겹친다. 발표에서 "Upstage가 파는 건 정확성 — 여기 우리 정확성 장치가 있다"로 연결하면 Upstage 기여도 점수에 직결.

06 환각을 막는 4겹

차별성의 핵심

이 서비스의 주장은 하나다 — "모델이 말한 것을 데이터가 검증한다." 원칙은 넷 다 같다: 근거가 없으면 그 문장을 채택하지 않는다.

검	하는 일
① 검수 게이트 AI는 후보, 사람이 통과	검수=확인이 아닌 행은 API가 안 내보낸다. 제보로 들어온 것도 예외 없다.
② 근거 판정 생성·검증 다른 모델	pro2가 이유를 쓰면 mini가 실제 메뉴·가격·도보와 대조해 grounded / notGrounded / notSure 판정. 근거가 없으면 그 문장을 채택하지 않고 데이터 사실 문장으로 바꾼다.
③ 기억 검증 없는 낱말 버림	"마라탕버거"를 검색해도 기억 안 함. 데이터 안에 실제로 있는 낱말만 남기고, 꺼낼 때 또 대조.
④ 추천 칩 검증 빈 결과 제안 금지	추천 아래 "도보 3분 안", "예산 1만원" 같은 후속 제안 버튼(칩)을 띄우기 전에, 서버가 그 조건으로 실제 검색을 한 번 돌려 결과가 1곳이라도 나오는지 확인한다. 빈 화면으로 보내는 칩은 안 띄운다(chips.js).

★ 시연 주의 — "교체 배지"는 거의 안 뜬다 (당황 금지)

생성 프롬프트가 애초에 근거 없는 표현(맛·평판·분위기)을 못 쓰게 막아서 대부분 통과한다(2026-07-24 실측: 15개 추천 전부 grounded, 교체 0건). 배지 안 뜨는 게 통제가 잘 됐다는 증거다. 억지로 띄우려 하지 말 것. 항상 보이는 근거 증거 = 추천 요약줄 ("추천 이유는 다른 모델이 데이터와 대조해 N건이 근거 검증을 통과했습니다"). "교체 장면"이 꼭 필요하면 /about.html 3단계 예시(초안→판정→교체)를 쓴다.

07

정확도와 재현성

깊이 15점

사진 5장·72개 메뉴를 사람이 채점한 결과 **55개 정확 = 76.4%**. 하지만 **정확도 숫자보다 무엇이 정확도를 흔드는지 알아낸 것**이 이 실험의 진짜 결과다.

사진 상태	정확도	의미
인쇄체 · 해상도 충분 (2장)	100% (37/37)	조건만 맞으면 완벽
표기 변형 · 손글씨 (2장)	64.0% (16/25)	중간
저해상 흐림 (1장)	20.0% (2/10)	여기서만 무너진다

지배 변수는 해상도 하나였다. 반사·거리는 문제가 아니었다 — 유리창에 붙은 메뉴를 멀리서 반사까지 없어 찍은 사진이 25개 항목 전부 맞았다.

→ 그래서 수집 지침이 "잘 찍어 달라"가 아니라 **"해상도만 확보하면 된다"**가 됐고, 저해상은 **사람 점수가 잡는** 구조로 만들었다.

재현성 — 이게 "깊이" 점수의 핵심

정확도 측정에 Document Parse를 **8번 호출**했고 비용은 약 **\$0.08**. 이때 **Parse가 돌려준 원본 응답(JSON) 5건을 파일로 저장해 뒀다.** 덕분에 우리 처리 코드를 고칠 때마다 API를 다시 부를 필요 없이 **저장해 둔 같은 응답으로 다시 채점**할 수 있었다 — 돈도 안 들고, 새로 부르면 결과가 조금씩 달라질 위험도 없다. 실제로 이 방식으로 **API를 한 번도 다시 부르지 않고 우리 코드만 두 번 고쳐 0% → 62.5% → 76.4%**로 올렸다.

1차 0건의 진실: 3장이 0건이었던 건 Document Parse 실패가 **아니라 우리 후처리 결함**이었다. OCR 텍스트가 `<figure><img alt="...">` 속성으로 오는데 초기 추출기가 태그를 통째로 버렸다. **alt 승격** 후 전 장의 텍스트를 확보했다.

08 시행착오 5선 — "깊이"를 증명하는 재료

걸린 것	어떻게 했나
사이드 메뉴 공기밥 1,000원이 "8천원 이하"를 통과시킴	메뉴마다 본메뉴/사이드를 판정해 예산 판정.개수 세기를 모두 본메뉴로만 자동 규칙 63케이스 + 사람이 예외만 뒤집음(아래 설명).
천원단위 소수 표기 가격을 7.0·1.5로 적는 식당	1,000배 환산. OCR 공백 천단위(4 000 , 21 ,000)도 같은 자리에서 흡수.
메뉴판을 그래프로 오인 "item_01 12,500원"이 화면에 나감	Parse가 그 구역을 "차트"로 표시 하고 메뉴 이름을 item_01 같은 기계 자동 이름 으로 붙였다. 우리 코드가 이 두 신호(차트 표시 + item_NN 이름)를 알아보고 그 줄을 걸러 시트에 넣지 않는다.
LLM 기억 오염 "마라탕버거"를 관심어로 저장	기억할 낱말을 실제 메뉴·식당 이름과 대조해 통과한 것만 남김. 꺼낼 때 또 대조.
추천 이유의 근거 Solar가 쓴 이유에 없는 말이 섞임	실제 데이터와 한 줄씩 대조. 대조는 다른 모델 이 맡고, 근거 없으면 데이터 사실로 교체.

사이드 판정은 "자동 + 사람" 2단이다(사람이 전부 손으로 뺀 게 아니다): ① **자동 규칙**(side-menu.js , 63케이스)이 **어느 가게에나 사이드인 것**을 먼저 걸러낸다 — 술·만두류·공기밥·100g 단가 표기·반접시. ② 그 규칙이 **틀린 줄만**, 검수하는 사람이 시트 비고 열로 뒤집는다 — 예: '모듬튀김 1인'(같은 집에 온전한 게 따로 있음), 'Green Salad'(같은 집 'Steak Salad'는 한 끼). 원칙: **어느 가게에나 통하는 것만 규칙에, 한 가게만의 예외는 사람이**. '튀김 1인'을 규칙에 넣었다면 다른 가게 본메뉴까지 잘못 끌려갔을 것.

09 데이터 구조 · 기술 스택 · 장애 대응

데이터 구조 — 정본은 구글 시트 하나 (DB 안 씀)

별도 DB 없이 **구글 스프레드시트 3탭**. 팀원 누구나 열어서 고치고, 검수도 시트에서 한다.

- **식당 (11열)**: 식당ID·이름·카테고리·주소·위도·경도·태그·메뉴판 사진 링크·수집자·촬영일·상태
- **메뉴 (7열)**: 식당ID·이름·메뉴명·가격·**출처**(사진파싱/제보/직접입력)·**검수**(확인/제외/대기)·**비고**(사이드/한끼)
- **학식**: 크롤러가 매일 채움. **마지막 들어온 날짜**를 함께 보내 "오늘 휴무"와 "정보 없음"을 구별.

기술 스택

층	선택 · 이유
프론트	바닐라 JS (빌드 없음) — 화면 5개·얇은 상태. 프레임워크 비용이 회수 안 됨
지도	Kakao Maps (ADR-0002)
데이터	구글 시트 단일 — 팀원이 바로 열어 고치고 검수도 여기서 (ADR-0005)
서버	Vercel serverless — API 키를 서버에만. 클라이언트 노출 없음
AI	Upstage Document Parse + Solar

장애 대응 — 하나가 죽어도 서비스는 돈다

멈추는 곳	대응
학식 크롤러	마지막 입력 날짜를 함께 보내 오늘보다 이르면 "정보 없음" (휴무와 구별)
Document Parse	제보 OCR 실패 시 직접 입력 으로 흐름 이어감
Solar	이유 생성 실패 시 데이터 템플릿 이유로 폴백 . 추천 루프는 안 죽는다
기능 격리	추천이 안 되어도 지도·학식은 보인다

검증(매 배포마다): 사이드 판정 63케이스 · 조건 추출 35 · 추천 칩 검증 18 · 배포 사이트 실브라우저 훑기 16(smoke). 결함을 일부러 주입해 smoke가 실제 FAIL 하는 것까지 확인(허수 테스트 방지). 정본 = `verification/matrix.md` .

10 한계 — 물으면 정직하게 (숨기면 더 약해진다)

한계	우리 대응
정확도 76.4%	저해상에서 크게 떨어짐 → 사람 검수를 필수 단계로
손글씨 침삭	5,000을 지우고 7,000 덧쓴 메뉴판은 실패 (인정)
범위 = 명륜캠 50곳	확장은 같은 파이프라인을 다른 동네에 (아직 안 함, 다음 후보 = 자연과학캠)
로그인 없음	기기 바꾸면 기억 안 이어짐 — 의도된 선택 (ADR-0004)
가격 = 촬영 시점 값	촬영일을 함께 저장하고 오래된 것부터 다시 확인

한계를 말하는 것 자체가 무기다 — **"안 한 것"을 말하면 신뢰가 오른다.** 학식 파싱 안 씬 · 순서 개인화 걷어냄 · 전량 자동 반영 안 함. 다 쓰는 것보다 **가려 쓴 게 세다.**

11 화면·API 전체 플로우 지도 (모르는 흐름이 없게)

공개 화면 4 + 비공개 검수 1 + cron 1. 각 화면은 **심사위원의 질문 하나**에 답한다.

화면	답하는 질문	부르는 API
/ 랜딩	이게 뭐고 왜 필요한가	config · data
/app.html 메뉴 찾기	예산 넣으면 진짜 정해주나	chat · recommend · parse-query (폴백) · data
/contribute.html 사진 제보	데이터는 어디서 오나 (내가 직접)	parse-menu (Doc Parse) · identify (Solar) · contribute
/about.html 파이프라인	왜 ChatGPT가 아닌가 + 지금 진짜인가	실행 블록 4개 (실제 API 호출)
/review.html 검수 (비공개)	AI가 뽑은 걸 사람이 승인	review (REVIEW_TOKEN)
cron 학식 크롤	매일 06:10 학식 자동 수집	cron/crawl-cafeteria

검색 경로 — 세 API가 하는 일이 다르다 (헛갈리면 안 됨)

① `/api/chat` = 문장을 넣으면 **가장 먼저 부르는 검색 두뇌**. solar-pro2가 문장을 조건(예산·도보·태그)으로 바꾸고, 대화 답변과 후속 제안 칩을 만든다. ② `/api/recommend` = 그 조건으로 실데이터에서 **Top3를 고르고**, Solar가 추천 이유를 쓰면 **solar-mini가 데이터와 대조해 채점**한다. ③ `/api/parse-query` = **비상용(폴백)**. chat이 느리거나 죽으면 AI 없이 정규식(패턴 매칭)으로 조건만 뽑아 검색을 살린다. 어느 길로 갔는지는 `source` 필드에 남는다.

제보 파이프라인 6단계 (검수 주체 = 운영진)

- ① `contribute` 메뉴판 사진 + 식당명 입력
- ② `parse-menu` Document Parse → {메뉴, 가격} 후보
- ③ 브라우저 파싱 결과 읽기 전용 확인
- ④ `contribute` 시트 `append` · 검수="대기"
- ⑤ `review` 운영진이 사진 원본과 대조 → 확인/제외 ← 사람
- ⑥ 서비스 검수="확인" 행만 노출

★ 중복 제보 처리 — "공개 제보면 데이터 오염되지 않나?"의 답

공개 제보라 같은 메뉴가 반복 들어온다. 접수(`contribute`)가 그 가게의 기존 행 + 이번 배치를 대조해 나눈다:
같은 이름 + 같은 가격 → 검수 "중복"(서비스·대기열에서 빠지되 흔적은 남김) · **같은 이름 + 다른 가격 → "대기"**
(가격이 바뀐 것일 수 있어 사람이 최선을 정함, 검수화면에  기존 확인가 N원 배치) · 새 이름 → "대기".
"중복"은 `검수=확인` 게이트를 못 넘어 서비스에 안 나온다.

12 심사 루브릭 100점 + 우리 답 + 증거

데이터 처리 35점(파이프라인 20 + 깊이 15) + Upstage 20점 = 55점에 발표 시간의 절반 이상을 쓴다. UI 완성도는 직접 배점 0 — Impact·발표로만 간접.

심사 기준	배점	우리 답 · 증거
범용 LLM 대비 차별성	5	vs표(3쪽)·환각 4겹(8쪽). "웹 가격은 판별 불가, 우리가 근거 데이터를 만듦"
문제 정의 구체성·타깃	5	명륜캠 학생, 하루 두 번 (2쪽)
정보 처리 파이프라인 설계·관리	20	입력 3갈래→검수 한 문→서비스, 모든 행이 출처·검수·촬영일 보유 (5·6쪽 / <code>/about.html</code> 흐름도 + 시트 실물)
정보 처리의 깊이·정교성	15	환각 4겹·정확도 해석(해상도 지배변수)·시행착오 (8·9·10쪽 / <code>verification/matrix.md</code>)
목적에 적합한 Upstage 적용	10	4자리 표 + "학식엔 안 씀"(가려 쓴 판단) (7쪽)
기술 적용 결과의 기여도	10	모델 선택 실측(mini 환각→pro2)·응답 보관 재현성 0%→62.5%→76.4%
실용성·활용 가능성	10	라이브 시연 (11쪽 / <code>sixsense.askewly.com</code>)
서비스 논리의 타당성	5	문제→해법→검증 일관 (2·3쪽)
임팩트·확장성	5	같은 파이프라인, 자연과학캠 확장 (13쪽)
발표 전달력·태도	5	리허설·분담 (7/25 11:00)
발표 자료 논리·완성도	10	14장 전체 (문제→해법→파이프라인→검증→실측→시연→한계)

승부처 한 줄: "그냥 ChatGPT에 물어보면 되지 않나?"에 실물(사진 유래 가격·당일 학식·근거 검증)로 답한다. 파랑 강조 4행 = 55점, 여기가 전부다.

윤영진이 말한 "기대 깊이" 3축에 우리 답 맞추기 (킵오프 p108)

새로 하는 게 아니라, 이미 하는 걸 채점 어휘로 라벨하는 것. 발표·Q&A에서 이 세 단어를 그대로 쓰면 심사 체크리스트에 정확히 걸린다.

윤영진 기대 축	우리가 이미 하는 것
데이터 융합 여러 소스 교차·종합	입력 3갈래(메뉴판 가격 + 학식 HTML + 주소·도보)를 한 검색 인터페이스로 교차
Action 기능 조화·요약 넘어 비교·추천·자동작성	예산으로 50곳 비교 + 추천 + 추천이유 자동작성
복합 구조 해석 표·수식·이미지 포함	메뉴판 이미지→구조화. 차트 오인·천원단위 소수 사고가 그 정교함의 증거

13 적대적 Q&A 방어 — 심사위원이 파고들 지점

각 카드: **합정**이 무엇이고, **답**은 무엇이며, **금지**는 무엇인지. ★ = 반드시 나온다.

★ 전제 — 심사위원은 Upstage 현직 엔지니어다 (키크오프에서 확정)

키크오프 자료가 "심사는 업스테이지 현직자들을 모셨습니다"라고 명시했다(추정 아님). 그래서 **역지로 끼운 API는 즉시 들킨다** — 반대로 우리의 "학식엔 Parse 안 씀".정직성 각주(Groundedness 전용모델 종료)가 오히려 가점이다. 부풀리지 말고, 가려 쓴 판단을 자신 있게 말한다.

"그냥 ChatGPT랑 뭐가 다르죠?" ★ 승부처

답 범용 챗봇은 **오늘자 학식을 모르고**, 가격은 찾아도 출처·시점이 뒤섞인 값을 준다(같은 집이 소스마다 6천/7천). 우리는 1차 소스만 — 학식은 **오늘 크롤링한 HTML**, 가격은 파싱 후 **사람이 승인한** 데이터. 행마다 출처·촬영일·검수자가 붙고, 모델이 한 말은 다른 모델이 데이터와 대조한다.

"요즘 ChatGPT는 웹검색으로 가격 찾잖아요?" ★ 대비 필수

합정 "웹에 가격 없다"고 했다가 반박당하는 지점.

답 맞다, **한 가게는 찾는다**. 그런데 실제로 검색해 보면 같은 짜장면이 다이닝코드 6,000원·블로그 7,000원 — 어느 게 지금 값인지 판별할 근거가 검색엔 없다. 우리는 촬영일 붙은 사진이 근거고, 바뀌면 제보→검수로 갱신. 그리고 "8천원 이하 되는 곳 전부"처럼 50곳을 필터·비교하는 질의는 실시간 검색으로 안 된다 — 구조화 데이터가 있어야 한다.

"정확도 76%면 낮지 않나요?"

답 그래서 **사람 검수를 필수로** 뒀다. 76.4%는 평균이고 인쇄체·해상도 충분이면 100%. 무엇이 정확도를 흔드는지(해상도) 알아낸 것이 이 실험의 결과다.

금지 "곧 올라갈 거예요" 식 변명 금지. 검수로 잡는 구조라고 답한다.

"거지맵이랑 뭐가 다른가요?" (숨기지 않는다)

답 표면은 닮았다(예산 필터+대화가). 차이는 **데이터 생산 방식** — 그쪽은 사용자가 손으로 타이핑, 우리는 사진 1장→Document Parse→프리필(\$0.01/장, 76.4%). **학식 자동 크롤링**과 상황 자연어 검색("과선배가 사는 날")도 우리만 있다.

"파싱이 틀리면? 데이터 오염은?"

답 전량 자동 반영을 **안 한다**. 파싱은 후보 프리필까지만, **운영자 승인**을 거쳐야 서비스에 나간다. 저해상 가격 오독(7.0→17.0) 실사례를 확보했고, 어디까지 자동인지의 경계를 **감이 아니라 실측으로** 그었다.

"공개 제보면 아무나 넣는데 데이터 오염 안 되나요?" ★ 파고들기 쉬움

답 두 겹으로 막는다. ① 제보는 전부 **검수="대기"**로 들어와 운영자 승인 전엔 서비스에 안 나온다. ② 접수 단계가 같은 가게 기존 행과 대조해 **같은 이름+같은 가격은 "중복"**으로 자동 격리(서비스 제외), **같은 이름+다른 가격은 "대기"**로 남겨 사람이 최신 가격을 정한다. 흔적은 **출처="제보"**로 남아 롤백도 된다.

"왜 학식엔 Document Parse를 안 썼나요?"

답 학식은 이미 **구조화된 HTML**. 문서 파서를 끼우는 건 요건용 억지 적용이고 "목적에 적합한 제품 적용"에 반한다. **비정형=Parse, 정형=크롤러** — 이 구분 자체가 우리의 Upstage 활용 설계다.

"전용 Groundedness Check 모델을 쓴 건가요?" ★ 정직성

합정 "제품 3종 썼다"고 부풀리면 바로 들킨다 (심사석에 Upstage 엔지니어 있다고 가정).

답 전용 모델은 2026-07 현재 계정 모델 목록에서 **제공 종료**(/v1/models 실측). 같은 계약(context-answer 판정)을 **생성과 분리된 solar-mini 판정자**로 구현했다. 정확한 서사 = "Solar를 생성·검증 2역할로 활용".

"Solar가 느리거나 죽으면?"

답 실측 **0.7~1.3초**, 타임아웃 시 **정규식 풀백** — 검색은 외부 장애와 무관하게 항상 동작하고 **source** 필드로 경로를 투명하게 남긴다. 모델 선정도 실측: solar-mini는 예산 환각·인원 나눗셈 실패를 보여 pro2를 채택했다.

"근거 검증이 실제로 뭘 걸러냈나요?"

답 생성 프롬프트가 근거 없는 표현을 애초에 막아 **대부분 통과**한다 — 검증은 그 통과를 **증명하는** 장치다. 프롬프트를 풀면 solar-mini가 맛·평판 문장을 잡아 데이터 팩트로 교체한다(/about.html 3단계 예시에 항상 그려져 있음).

"왜 DB 대신 시트인가요?"

답 데이터 수집·보강이 **팀원 4명의 협업**이라서다. 시트는 계정·어드민 없이 누구나 열어 고치고 예외 처리 (비고 열)까지 그 자리에서 된다. 검수 페이지도 시트를 정본으로 읽고 쓰는 창일 뿐. 지금 규모(700여 행)엔 시트가 운영 비용 최소, 커지면 DB로 옮기면 된다 — **읽기 경로가 한 곳이라 교체도 쉽다**(ADR-0005).

"확장 가능한가요?" / "이게 진짜 도나요?"

답 **확장:** 구조가 이식 단위다 — 학식 크롤러(대학별 URL 교체)·메뉴판 파싱(지역 무관)·예산 질의(공통). 자연과학캠이 다음 후보. **진짜 도냐:** 라이브에서 바로 보여준다(/about.html 실행 블록이 실제 API를 그 자리에서 호출). 정본 = `verification/matrix.md`.

"왜 Parse만? Classify-Extract는 안 썼나요?" ★ 현직자 예상

합정 "Extract 쓸 줄 몰라서"로 들리면 감점. "필요 없어서, 우리가 더 정확히 했다" 프레임으로.

답 메뉴판은 Parse의 **좌표+텍스트만으로 충분**하다. Extract가 할 '필드 추출'(가격·메뉴명 정리)은 우리가 **도메인 특화 후처리 규칙**으로 직접 했다 — Parse가 준 날것을 **메뉴판이라는 우리 문제에만 맞춘 규칙 코드**로 다듬는 단계다(7.0→7,000 환산 · 4 000 공백 흡수 · 공기밥=사이드 63케이스 · 차트 오인 줄 걸러냄). 범용 추출 모델보다 이 도메인에선 우리 규칙이 더 정확·투명하고 비용 0. 학식과 같은 **목적 적합성** 판단.

"kickoff에서 배운 Supabase·Next.js는 왜 안 썼나요?"

답 배운 걸 못 쓴 게 아니라 **문제 크기에 맞춰 골랐다**. 화면 5개·얕은 상태라 프레임워크 밀키트 비용이 회수 안 된다. 데이터는 팀 4명 협업·검수라 **시트가 곧 어드민**(계정·어드민 도구 없이 누구나 고침). 규모 커지면 DB로 옮기면 된다 — 읽기 경로가 한 곳이라 교체도 쉽다(ADR-0005).

답 못 할 질문이 나오면

지어내지 않는다. **"정직한 한계" 프레임**으로 받는다 — "그건 아직 안 했다 / 지금은 여기까지가 실측이다." 없는 걸 있다고 하는 순간 나머지 답까지 의심받는다.

정본: `docs/SERVICE-SPEC.md` (§1~§12). 숫자는 항상 §8 기준. 발표 멘트는 `docs/presentation/slides-script.md`. 이 문서는 2026-07-24 실측을 발표 공부용으로 재구성한 것. · TEAM SIXSENSE